

Trabalho Prático – Etapa 1 – GCC130

Analizador Léxico (Flex)

Caio Bueno Finocchio Martins
Lana da Silva Miranda
Nina Tobias Novikoff da Cunha Ribeiro

Universidade Federal de Lavras (UFLA)
Professor: Ricardo Terra

1 Introdução

A análise léxica corresponde à primeira fase do processo de compilação. Nessa etapa, o código-fonte é lido e suas sequências de caracteres são agrupadas em unidades léxicas, chamadas tokens, que representam elementos básicos da linguagem. Para isso, utilizam-se padrões descritos por expressões regulares, permitindo que o analisador reconheça lexemas válidos e associe a cada ocorrência uma ação específica. Além de preparar a entrada para as próximas fases do compilador, essa etapa também pode registrar identificadores em uma tabela de símbolos e indicar erros léxicos com sua respectiva posição no texto.

Neste projeto, o objetivo da Etapa 1 foi implementar, com o uso do Flex, um analisador léxico para uma mini linguagem didática. O analisador desenvolvido foi responsável por reconhecer declarações de tipos, identificadores, números inteiros e decimais, palavras-chave, operadores relacionais, aritméticos e lógicos, além de símbolos de pontuação. Também fez parte do escopo imprimir o token reconhecido, seu lexema e sua posição no arquivo de entrada, desconsiderar espaços em branco e comentários, detectar e reportar erros léxicos ao usuário e exibir a tabela de símbolos construída durante a análise.

2 Decisões de Projeto

As seguintes decisões de projeto foram tomadas para otimizar o desempenho, a funcionalidade e a robustez do analisador léxico:

- **Inclusão de Tokens Adicionais:** Para aumentar a funcionalidade da linguagem e facilitar validações futuras, foram adicionados tokens além dos requisitos mínimos. O operador de atribuição (=) foi incluído para permitir operações de atribuição de valores, e literais (cadeias de caracteres entre aspas) foram definidos para suportar a representação de dados textuais.
- **Estrutura de Dados para a Tabela de Símbolos:** Optamos por implementar uma tabela hash com listas encadeadas para gerenciar a tabela de símbolos. Essa escolha se baseou na eficiência da busca e inserção de símbolos. O uso de listas

encadeadas foi adotado para tratar colisões, garantindo a integridade e acessibilidade dos dados, mesmo em cenários com alta ocupação da tabela.

- **Tratamento de Palavras Reservadas:** As palavras reservadas da linguagem foram tratadas diretamente por meio de diagramas de transição no analisador léxico, sem serem armazenadas na tabela de símbolos. Essa abordagem evita buscas desnecessárias na tabela para elementos fixos da linguagem, otimizando o reconhecimento léxico e tornando mais fácil distingui-las dos identificadores definidos pelo usuário.
- **Categorização de Tokens e Uso de `yylval`:** Os tokens foram definidos em categorias gerais (por exemplo, `OPERADOR_RELACIONAL`, `OPERADOR_ARITMETICO`), e o campo `yylval` foi usado para diferenciar lexemas que pertencem à mesma categoria. Por exemplo, "`<=`" e "`>`" são ambos `OPERADOR_RELACIONAL`, mas seus `yylval` são `OR_LE` e `OR_GT`, respectivamente. No entanto, o operador de atribuição (`=`) e as palavras reservadas possuem tokens individuais para cada um, sendo exceções à regra adotada.
- **Uso de `typedef union` para `yylval`:** Para permitir que `yylval` armazenasse diferentes tipos de dados conforme o lexema analisado, usamos um `typedef union`. Essa estrutura permite que `yylval` adote o tipo de um inteiro, ponto flutuante, caractere ou um ponteiro para a estrutura `Simbolo`, conforme a necessidade do token.
- **Definição do `yylval` por Tipo de Lexema:**
 - **Identificadores:** `yylval` armazena um ponteiro para a estrutura `Simbolo` (`*Simbolo`).
 - **Operadores Lógicos, Aritméticos, Relacionais e Tipos de Dados:** `yylval` armazena um valor inteiro (`int`) definido por `#define` que representa cada lexema específico.
 - **Delimitadores:** `yylval` armazena o caractere do delimitador (`char`).
 - **Números Inteiros:** `yylval` armazena o valor inteiro (`int`).
 - **Números Ponto Flutuante:** `yylval` armazena o valor de ponto flutuante (`float`).
 - **Literais:** Nesta etapa, `yylval` não foi definido para literais. Decidiu-se adiar o tratamento do valor semântico de literais para a análise sintática, devido ao problema de vazamento de memória.
- **Tratamento de Erros Específicos com Expressões Regulares:** Foram usadas expressões regulares para identificar e relatar erros léxicos específicos, como comentários de múltiplas linhas não fechados, literais que não terminam em aspas, números com zero à esquerda (`012`, `-06`), números de ponto flutuante com vírgula em vez de ponto (`1,23`), e identificadores que não começam com o caractere `$`. Essa abordagem permite mensagens de erro mais claras para os desenvolvedores.

3 Dificuldades Encontradas

Durante o desenvolvimento do analisador léxico, surgiram desafios que afetaram as decisões e a implementação do projeto. As principais dificuldades incluem:

- **Distinção entre Decisão de Projeto e Expectativas da Etapa:** Foi necessário discernir se algumas abordagens eram escolhas deliberadas de projeto ou se representavam um desvio das expectativas da fase de análise léxica.
- **Compreensão da Função da Tabela de Símbolos na Análise Léxica:** A função da tabela de símbolos na fase léxica gerou questionamentos, pois, embora seja essencial para o compilador como um todo, sua interação e responsabilidades nessa etapa precisaram ser pesquisadas a fundo para evitar erros por falta de entendimento.
- **Diferenciação no Tratamento de Erros (Léxico vs. Sintático):** Foi importante estabelecer uma clara fronteira entre os erros tratados pelo analisador léxico e aqueles que eram responsabilidade do analisador sintático, de forma a não ignorar erros léxicos nem incluir erros sintáticos nessa etapa.
- **Definição do `yylval` para Diferentes Lexemas:** Atribuir o valor semântico (`yylval`) adequado para cada lexema, considerando a variedade de tipos de tokens, foi um desafio técnico. Era necessário um mecanismo flexível para armazenar diferentes tipos de dados, como inteiros, `float`, ponteiros e caracteres, o que exigiu uma solução robusta e eficiente.

4 Diagramas de Transição (DFAs)

4.1 DFA 1 – Literais

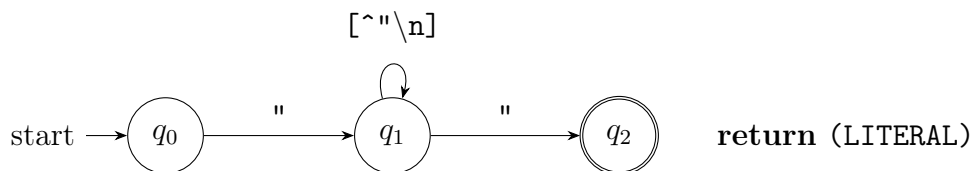


Figura 1: DFA para reconhecimento de literais.

4.2 DFA 2 – Identificadores

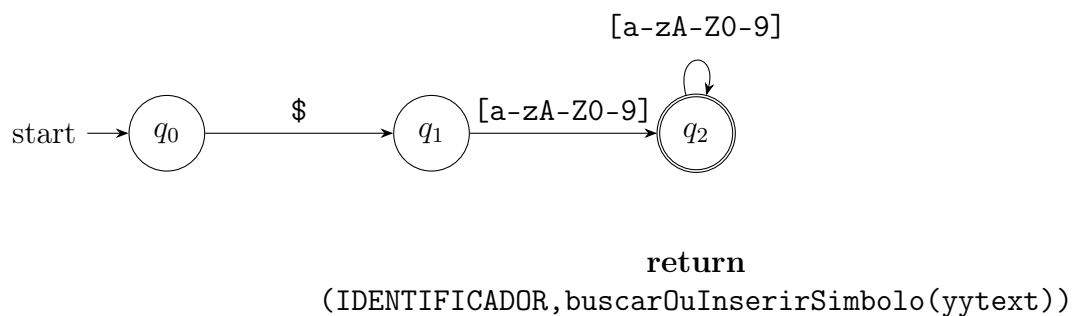


Figura 2: DFA para reconhecimento de identificadores (padrão `$[a-zA-Z0-9]+`).

5 Arquivo de Teste e Saída Esperada

5.1 Arquivo de Entrada (teste_lexer.1)

```
1  /* Comentário de múltiplas linhas
2  fechado corretamente */
3  // Comentário de uma linha
4
5  int $x = 10;
6  float $y = 25.5;
7  bool $flag = true;
8  bool $flag2 = false;
9
10 $x = $x + 1;
11 $y = $y / 2.0;
12 $flag = ! $flag;
13
14 if ($x < 20 && $flag == true) {
15     print("texto literal fechado");
16 }
17 else {
18     read($x);
19 }
20
21 while ($x != 0 || $y >= 1.0) {
22     $x = $x - 1;
23     $y = $y * 3.5;
24     return;
25 }
26
27 ( ) { } ; ,
28 < > == <= >= !=
29 + - * /
30 && || !
31 =
32
33 123
34 45.67
35 -99
36 0
37
38 $x
39 $abc123
40 $A1
41
42 -0
43 001
44 -012
45 -3.14
46 12,34
47 abc
48 "literal aberto
49 @
50
51 /* comentário de múltiplas linhas não fechado
```

Listing 1: Arquivo de teste utilizado

5.2 Saída do Analisador (stdout)

	LIN	COL	LEXEMA	TOKEN
1				
2				
3	5	1	int	TIPO_DADO
4	5	5	\$x	IDENTIFICADOR
5	5	8	=	OPERADOR_ATRIBUICAO
6	5	10	10	NUMERO INTEIRO
7	5	12	;	DELIMITADOR
8	6	1	float	TIPO_DADO
9	6	7	\$y	IDENTIFICADOR
10	6	10	=	OPERADOR_ATRIBUICAO
11	6	12	25.5	NUMERO FLOAT
12	6	16	;	DELIMITADOR
13	7	1	bool	TIPO_DADO
14	7	6	\$flag	IDENTIFICADOR
15	7	12	=	OPERADOR_ATRIBUICAO
16	7	14	true	PALAVRA_RESERVADA
17	7	18	;	DELIMITADOR
18	8	1	bool	TIPO_DADO
19	8	6	\$flag2	IDENTIFICADOR
20	8	13	=	OPERADOR_ATRIBUICAO
21	8	15	false	PALAVRA_RESERVADA
22	8	20	;	DELIMITADOR
23	10	1	\$x	IDENTIFICADOR
24	10	4	=	OPERADOR_ATRIBUICAO
25	10	6	\$x	IDENTIFICADOR
26	10	9	+	OPERADOR_ARITMETICO
27	10	11	1	NUMERO INTEIRO
28	10	12	;	DELIMITADOR
29	11	1	\$y	IDENTIFICADOR
30	11	4	=	OPERADOR_ATRIBUICAO
31	11	6	\$y	IDENTIFICADOR
32	11	9	/	OPERADOR_ARITMETICO
33	11	11	2.0	NUMERO FLOAT
34	11	14	;	DELIMITADOR
35	12	1	\$flag	IDENTIFICADOR
36	12	7	=	OPERADOR_ATRIBUICAO
37	12	9	!	OPERADOR_LOGICO
38	12	11	\$flag	IDENTIFICADOR
39	12	16	;	DELIMITADOR
40	14	1	if	PALAVRA_RESERVADA
41	14	4	(	DELIMITADOR
42	14	5	\$x	IDENTIFICADOR
43	14	8	<	OPERADOR_RELACIONAL
44	14	10	20	NUMERO INTEIRO
45	14	13	&&	OPERADOR_LOGICO
46	14	16	\$flag	IDENTIFICADOR
47	14	22	==	OPERADOR_RELACIONAL
48	14	25	true	PALAVRA_RESERVADA
49	14	29	)	DELIMITADOR
50	14	31	{	DELIMITADOR
51	15	5	print	PALAVRA_RESERVADA
52	15	10	(	DELIMITADOR
53	15	11	"texto literal fechado"	LITERAL
54	15	34	)	DELIMITADOR
55	15	35	;	DELIMITADOR
56	16	1	}	DELIMITADOR

57	17	1	else	PALAVRA_RESERVADA
58	17	6	{	DELIMITADOR
59	18	5	read	PALAVRA_RESERVADA
60	18	9	(	DELIMITADOR
61	18	10	\$x	IDENTIFICADOR
62	18	12	)	DELIMITADOR
63	18	13	;	DELIMITADOR
64	19	1	}	DELIMITADOR
65	21	1	while	PALAVRA_RESERVADA
66	21	7	(	DELIMITADOR
67	21	8	\$x	IDENTIFICADOR
68	21	11	!=	OPERADOR_RELACIONAL
69	21	14	0	NUMERO INTEIRO
70	21	16		OPERADOR_LOGICO
71	21	19	\$y	IDENTIFICADOR
72	21	22	>=	OPERADOR_RELACIONAL
73	21	25	1.0	NUMERO FLOAT
74	21	28	)	DELIMITADOR
75	21	30	{	DELIMITADOR
76	22	5	\$x	IDENTIFICADOR
77	22	8	=	OPERADOR_ATRIBUICAO
78	22	10	\$x	IDENTIFICADOR
79	22	13	-	OPERADOR_ARITMETICO
80	22	15	1	NUMERO INTEIRO
81	22	16	;	DELIMITADOR
82	23	5	\$y	IDENTIFICADOR
83	23	8	=	OPERADOR_ATRIBUICAO
84	23	10	\$y	IDENTIFICADOR
85	23	13	*	OPERADOR_ARITMETICO
86	23	15	3.5	NUMERO FLOAT
87	23	18	;	DELIMITADOR
88	24	5	return	PALAVRA_RESERVADA
89	24	11	;	DELIMITADOR
90	25	1	}	DELIMITADOR
91	27	1	(	DELIMITADOR
92	27	3	)	DELIMITADOR
93	27	5	{	DELIMITADOR
94	27	7	}	DELIMITADOR
95	27	9	;	DELIMITADOR
96	27	11	,	DELIMITADOR
97	28	1	<	OPERADOR_RELACIONAL
98	28	3	>	OPERADOR_RELACIONAL
99	28	5	==	OPERADOR_RELACIONAL
100	28	8	<=	OPERADOR_RELACIONAL
101	28	11	>=	OPERADOR_RELACIONAL
102	28	14	!=	OPERADOR_RELACIONAL
103	29	1	+	OPERADOR_ARITMETICO
104	29	3	-	OPERADOR_ARITMETICO
105	29	5	*	OPERADOR_ARITMETICO
106	29	7	/	OPERADOR_ARITMETICO
107	30	1	&&	OPERADOR_LOGICO
108	30	4		OPERADOR_LOGICO
109	30	7	!	OPERADOR_LOGICO
110	31	1	=	OPERADOR_ATRIBUICAO
111	33	1	123	NUMERO INTEIRO
112	34	1	45.67	NUMERO FLOAT
113	35	1	-99	NUMERO INTEIRO
114	36	1	0	NUMERO INTEIRO

```

115 38 1 $x IDENTIFICADOR
116 39 1 $abc123 IDENTIFICADOR
117 40 1 $A1 IDENTIFICADOR
118 42 1 -0 Erro léxico: o valor zero não pode ser negativo
119 43 1 001 Erro léxico: 0 a esquerda
120 44 1 -012 Erro léxico: 0 a esquerda
121 45 1 -3.14 Erro léxico: pontos flutuantes não podem assumir
valores negativos
122 46 1 12,34 Erro léxico: pontos flutuantes devem usar "." e não ","
123 47 1 abc Erro léxico: identificadores devem começar com "$"
124 48 1 "literal aberto Erro léxico: literais devem ser fechados com aspas
125 49 1 @ Erro léxico: caractere inválido
126 52 1 /* comentário de múltiplas linhas não fechado
127 Erro léxico: comentários de múltiplas linhas devem ser fechados com "*/"
128
129
130 Tabela de Símbolos
131 HASH LEXEMA TOKEN QUANT. OCORRENCIAS
132 -----
133 16 $flag 18 4
134 24 $x 18 9
135 25 $y 18 6
136 40 $abc123 18 1
137 41 $flag2 18 1
138 98 $A1 18 1
139 -----

```

Listing 2: Saída gerada pelo analisador léxico

6 Conclusão

Este trabalho permitiu transformar conceitos teóricos da análise léxica em uma implementação concreta, tornando mais claro como o compilador começa, de fato, a interpretar um programa. Ao longo da etapa, foi possível observar que escolhas aparentemente simples influenciam diretamente a clareza da solução e a forma como o sistema poderá evoluir depois.

Mais do que chegar a um resultado funcional, esta etapa serviu para consolidar a lógica por trás do reconhecimento de padrões, da identificação de erros e da organização das informações obtidas durante a leitura do código-fonte. Com isso, foi possível concluir esta etapa de forma satisfatória, unindo prática e teoria no desenvolvimento do analisador léxico.